

Limits of Metamorphic Testing for Object-Level Authorization: False Proofs, Oracle Readability, and Measurement Resolution

Qiulong Lin^{a,b}, Nanlei Chen^{a,b,*}

^aYango University, Fuzhou, 350015, China

^bSogang University, Seoul, 04107, Republic of Korea

ARTICLE INFO

Keywords:

oracle problem
metamorphic testing
object-level authorization
BOLA/BFLA
specificity measurement
false proof

ABSTRACT

Metamorphic testing is the standard answer to the oracle problem in Web security testing: MST-wi defines 76 system-agnostic metamorphic relations (MRs) over a Web-specific domain language, and reports a specificity of 99.81%. We show that, for object-level authorization, these relations delegate adjudication to a *GUI-reachability model* that is a *crawl-time snapshot*, and that the delegation fails deterministically, in one direction.

From a three-phase instantiation of the predicates as originally defined—crawl, out-of-band authorization change, execute—a 576-record scenario/configuration study and a controlled study on Gitea 1.22.6, replicated on Gogs 0.14.3 and GitLab 17.11.7, characterise the failure as three boundaries. (i) An access legitimately granted out of band after the crawl is reported as a *proven* violation—a *false proof*—because one of the predicate’s two representations is unavailable. (ii) Repairing this with a readable, sound authorization oracle makes the MR step add no adjudication information there—a necessary condition, not a surprise: a direct adjudicator consuming the same oracle and request outcome reproduces the verdict, and replacing the step with an ordinary request changes nothing measurable (a descriptive point estimate, not a non-inferiority test, and not domination). (iii) The reported specificity is not conditioned on a reachability—authorization check and its input-level denominator cannot resolve a scenario-level failure.

On Gitea the mismatch appears only under a crawl-time reading and never live, locating the mechanism in *time*, not visibility; and the oracle that would discharge the condition is readable from an owner’s vantage but not a black-box subject’s. We confirm the predicates’ semantics at source and execute the published relations—source unmodified, build configuration and harness added—reproducing the false proof on a real system within the reachability-gated subset that fires in our run.

1. Introduction

Broken object-level authorization (BOLA), first in the OWASP API Security Top 10 [1], is the failure to enforce, per object, that the authenticated subject is entitled to the object touched. Detecting it is an *oracle* problem: vulnerable and correct systems answer with the same success status. The standard answer is metamorphic testing, and MST-wi [2] is its most developed instance for Web systems: 76 system-agnostic relations (MRs) over 55 Web-specific predicates, reported to detect 85% of vulnerabilities with specificity 99.81%.

This paper asks what happens to that claim when the asserted property is object-level authorization. The predicates do not express entitlement; they express what their authors call reachability—what “the functionality exposed by the user interface” permits—with the role-to-resource mapping “**not always available**” [2]. We take that sentence as premise, not finding. Delegating adjudication to a reachability model makes the moment the model was read part of the specification, so we instantiate the predicates *as defined*—over a model recorded at crawl time—and drive the system through three phases: crawl, change authorization out of band, execute. The middle phase is the one the original evaluation does not exercise.

We ran a 576-record scenario/configuration study on a constructed system and a controlled study on Gitea 1.22.6, replicated on Gogs 0.14.3 and GitLab 17.11.7. We propose no mechanism; an earlier iteration of this work did, and our own ablation removed it. What survives is a characterisation as three boundaries (Section 6): (i) the adjudication as designed produces *deterministic false proofs*—a legitimate access granted out of band after the crawl is reported as a proven violation, at rate 1.00 on the constructed system and 0.4000 on Gitea, because the predicate’s two representations cannot describe an out-of-band grant; (ii) when a sound, readable authorization oracle is available, the metamorphic step carries no adjudication information the oracle-plus-request pair does not—a necessary condition (Lemma 1), not an empirical surprise, whose empirical face is that replacing the step with a single ordinary request changes nothing measurable (false proofs 0 vs. 0, recall 0.50 vs. 0.49; a descriptive, point-estimate comparison, not a formal hypothesis test, no margin pre-specified, not domination), and on all 3 disagreeing records the MR-based method errs; (iii) the reported specificity is not conditioned on an explicit check that the reachability model coincides with the authorization state, and is computed in units insensitive to scenario-uniform failure. These are organised as RQ1 (temporal), RQ2 (oracle), RQ3 (measurement); the native execution, the seeded target and the second implementation are support evidence for them, not further headline findings.

*Corresponding author.

✉ ubfbzu3@163.com (N. Chen)

The boundaries are not a claim that MST-wi is wrong: the reachability stand-in is announced by its authors, and the predecessor design attributes its residual false alarms to crawler coverage [3]. Our contribution is to locate the failure mechanically—in *when* the model was read, not *which* objects are visible—and to attach it to the reported metric: a three-phase instantiation with internal controls (Section 3); the scenario/configuration study (Section 4); the Gitea study with an endogenous control, a seeded-vulnerability positive control, and a native execution of the library’s own relations attributing the false proof to the published code (Section 5); and the boundaries with an admissibility condition and a formal statement of the metric’s resolution (Section 6).

2. Background: the predicates in question

For authorization there is no intrinsic oracle: vulnerable and correct executions both answer with success. Metamorphic testing replaces the absent oracle with a relation over multiple executions—and the relation is only as sound as the assumptions it needs, which for authorization are exactly the facts the oracle was supposed to supply. MST-wi [2], building on Mai et al. [3], crawls a per-user model of reachable URLs and contents, then checks MRs against that recorded model. In the artifact we inspected, the predicates guarding an authorization MR read either the recorded model directly (`cannotReachThroughGUI`, `userCanRetrieveContent`) or static subject properties; we call a predicate *GUI-derived* when its value is a function of the crawl.

Why the false proof is a design consequence, not a bug
A single example makes the mechanism concrete. Suppose a subject can reach an object at crawl time, so the record stores that object as reachable. Afterwards an administrator grants the same subject access to a *further* object through a channel—an administrative endpoint or a share link—that a crawl of the subject’s own interface never traverses. At execution the guard still reads the frozen record, finds the new object absent from it, and concludes the subject cannot reach that object; the relation then treats the successful response as a violation. Every step is exactly as specified, so no implemented guard is at fault and no individual relation is miswritten. The false proof is structural: the predicate’s two representations—“reachable as recorded” and “not reachable as recorded”—cannot express “reachable, but only after the record was taken”. More crawling does not remove it, because the grant need not be visible through the subject’s interface at all; and the reported specificity never registers it, because the error is a property of the adjudication’s *reading moment* rather than of any single generated input. This is the mechanism the rest of the paper measures, and the reason we speak of a boundary of the specification rather than a defect of an implementation.

Two mechanical counts frame what follows. Over the 76 catalog MRs, 38 (50.0%) gate on a GUI-derived or identity predicate, and only 14 (18.42%) are authorization-relevant by name or action—an upper bound, reported as a caveat. And a full-text search for predicates reading the *state*

of an authorization artifact (ownership, permission record, share) finds 0 occurrences. The delegation to reachability is therefore unavoidable, and the reading moment enters the specification: if an entitlement changes after the crawl, the MR evaluates reachability against a world that no longer exists. This is a statement about what the relations *mean*; our experiments are built so that the crawl-time and live readings can be told apart.

3. Protocol: a faithful three-phase instantiation

Two instruments carry distinct claims. The three-phase instantiation below carries the quantitative results of Sections 4 and 5; a direct execution of the library’s own relations—source unmodified, with the build configuration and a replay harness added—carries one qualitative claim, that the published code produces the false proof we predict (Section 6.5). Neither instrument is a benchmark: we report no sensitivity or specificity attributable to MST-wi.

Three phases (1) *Crawl*: record, per subject and object, whether the subject can reach the object through the interface, and freeze that record—the model the predicates will consult. (2) *Change authorization out of band*: alter a (subject, object) entitlement through a channel outside the crawled interface; the frozen model is not updated, because the architecture has no mechanism to update it. (3) *Execute the MR* against the current state while the predicates consult the frozen model. Phase 2 is what the original evaluation does not exercise, and it is where the reachability stand-in can be wrong.

Adjudicator variants and internal controls A raw alarm becomes a verdict through an exception policy, and different policies are different methods: **v1** (raw alarms proven); **v2** (legitimate-sharing exceptions filtered); **v3** (state-calibrated—consults a readable authorization introspection surface, abstains when none is readable); `degrade_all` (the floor: abstain on every alarm); `direct_only` (authorization surface only, access outcome ignored—a deliberately unfair control); and `direct_observe` (a succeeding plain request checked against the same readable surface). The last consumes the same observable quantity as the MR family and decides the negative result.

Scenarios and design cells Seven scenarios separate the reasons an MR can misfire: no shared object (S1, S2); an in-band legitimate share visible in the framework log (S3); an unshared plain violation (S4); a read-only share plus a write violation (S5); an out-of-band share invisible to the framework log (S6); an out-of-band share plus a genuine violation (S7). They are crossed with 8 configurations defined by instantiation mode, enforcement correctness, channel visibility and introspection availability. The nominal cross is $7 \times 8 = 56$ combinations, but four are inadmissible: the two no-exception configurations (`V-noE3-vuIn0`, `V-noE3-vuIn1`) are undefined under the two out-of-band scenarios (S6, S7),

where the legitimate-sharing exception does not apply. This leaves 52 admissible design cells.

Statistical treatment The 576 records are not independent observations. Each admissible cell is expanded deterministically by the factors that are actually varied: 2 metamorphic relations (MR-002, MR-004), 3 exception-policy variants (raw, one level, two levels—the first deliberately thrown switch), and 2 reachability readings of the frozen model (crawl-time blind, crawl-time aware—the second deliberately thrown switch). The two relations do not both apply to every cell: only MR-002 applies under scenario S5, so 8 S5 cells carry one relation while the remaining 44 carry two. The record count is therefore

$$576 = 44 \cdot (2 \cdot 3 \cdot 2) + 8 \cdot (1 \cdot 3 \cdot 2),$$

a designed study, not a balanced factorial—the per-cell multiplier differs by scenario. All intervals (Wilson, or Newcombe for differences; our own implementation in `compute_all.py`) are therefore *descriptive*—magnitude and direction only, no formal hypothesis-test reading—and no conclusion rests on statistical significance. The replication ceiling is explicit: one constructed system, 52 cells, 2 MRs, three real systems. Where a headline count has a small denominator (the anchored rate, $2/2$, is wide by construction) we lead with the larger-denominator count of the same phenomenon and never quote a small-denominator rate without its interval.

Tooling and code provenance Development of the analysis scripts was assisted by an AI coding assistant (declared in Section 9). Every script that assistant touched was reviewed, executed, and its output checked by the authors against the raw records; the released pipeline recomputes every reported scalar from those records with no hand-entered value in the body. No AI tool participated in the detection loop or in any adjudication.

4. Synthetic study: where the adjudication goes

4.1. Definitions

Confusions are defined over the *verdict*: a *false proof* is the strict subset of false alarms promoted to violation-proven—an error the tool asserts as established. We also report the *detection-keep rate* (share of fired true violations still alarmed) and count only abstentions justified by an unreadable oracle.

4.2. What the constructed system shows

Attribution is clean On the anchored configuration the false-proof rate is $2/2 = 100.0\%$ ([34.24%, 100.0%]); with the legitimate-sharing exception absent the rate is 0.00 (0/2), and with the channel listed it is 0.00 (0/2). The phenomenon is therefore attributable to a legitimate sharing exception the MR cannot know about, not to execution itself. A 2×2 grid over channel visibility and enforcement correctness tells

the same story from the other side: on unlisted cells the false-proof rate is 44.44% with no false negatives; on listed cells it drops to 0.0% but false negatives appear (1 of 4+1 on the violating cell). Avoiding false proofs and keeping detection are controlled by one switch, and the switch cannot be thrown both ways at once.

Filtering exceptions does not help The exception ladder—raw, one exception level, two—leaves the anchored rate at 1.00, 1.00, 1.00: it does not move. The exception levels we can express are themselves functions of the same reachability model, so they inherit its blind spot.

The framework cannot see the channel that matters

When a legitimate share was created through the framework's facility, its log holds 12 records and v2 produces 0 false proofs (rate 0.00); when the same share was created out of band, the log holds 0 records—no evidence the share exists—and v2's false proofs rise to 8 of 8 alarms (rate 1.00), every one promoted to violation-proven. The MR cannot distinguish “this share does not exist” from “this share exists but was not crawled”; its only representation of the first is the second. On S7, where a genuine violation *does* sit behind the out-of-band share, the same blindness makes the alarm right for the wrong reason (6 of 6 reached)—and the coincidence of alarms with violations there is exactly what an input-level metric cannot unpack. The same tension appears within a single MR: for S3 (must not alarm) vs S5 (must alarm), no configuration in the family is correct on both—the choice of switch is a choice of *which* error to make.

Stratification, not aggregation Aggregated, v3 looks best (recovery 1.00 against `degrade_all`'s keep rate 0.00); the reading is wrong, because v3 changes regime with oracle readability. Stratified: on the readable stratum (36 records) it removes v2's false proofs (0 vs. 4) while still alarming on all 9 fired true violations; on the absent stratum (18) it alarms on none of the 5 true violations v2 still alarms on, abstaining 13 times. Pooling converts honest abstention into apparent performance, so v3's numbers may appear only stratified.

4.3. The decisive ablation: does the metamorphic step carry work?

Our earlier iteration proposed v3 as a mechanism, resting on the claim that the metamorphic step was necessary. We tested that claim by removing the step and substituting the most naive consumer of the same observable. The distinction matters: `direct_only`—oracle yes, access outcome ignored—produces 108 false proofs against v3's 0 (precision 0.50), and rejecting it proves nothing because it discards half the evidence. The fair control is `direct_observe`: a succeeding ordinary request plus the same readable surface, consuming exactly what the MR-based adjudicator consumes.

Table 1 closes this line of work. `direct_observe` has 0 false proofs against v3's 0—a tie at zero, not a win—with the same point-estimate precision (1.00), and recall 0.50 against 0.49. The recall difference of 0.0139 is not distinguishable

Table 1

End-to-end confusion matrix over all 576 records (216 true violations, 360 correct cases). Intervals are Wilson 95% and are descriptive (Section 3). `direct_only` is a deliberately unfair control, shown to make that point.

method	TP	FP	FN	TN	precision	recall	F1
v1 (raw)	198	60	18	300	0.77	0.92	0.84
v2 (exception-aware)	213	60	3	300	0.78	0.99	0.87
v3 (state-calibrated)	105	0	111	360	1.00	0.49	0.65
degrade_all	0	0	216	360	—	0.00	—
direct_only	108	108	108	252	0.50	0.50	0.50
direct_observe	108	0	108	360	1.00	0.50	0.67

from zero (95% CI $[-0.0797, 0.1072]$): a descriptive, point-estimate comparison, not domination; no margin was pre-specified, so no formal hypothesis-test claim attaches to it. We found no measurable advantage of the metamorphic step over direct observation in these oracle-available scenarios, and that absence is decisive: the step would have to be better at something the plain request cannot do, yet on every one of the 3 disagreeing records it is v3 that fails, its reachability precondition blocking a detection the plain request makes. The comparison is therefore a descriptive, paired one: only three records differ, and all three favour `direct_observe`—on each, v3 abstains or misses while the plain request raises the correct alarm. Selective presentation would have hidden the cost: v3’s precision of 1.00 was bought with 111 false negatives (F1 0.65), an exchange of 60 false alarms for missed violations dressed as an improvement.

4.4. Reproducing the reported specificity, and why it cannot see this

The reported specificity has the form

$$S_{\text{input}} = 1 - \frac{\text{false alarms}}{\text{inputs}}.$$

We cannot reproduce the engine’s exact input count, so we approximate two follow-up inputs per case—enough to exhibit the denominator’s resolution, the only point at issue. In that approximation the configuration in which a whole class of legitimate access is misreported still yields a specificity of 0.846154 (84.62%). A failure uniform across a scenario class is diluted by every other input in the run, and the predecessor design says the same of itself, attributing its residual false alarms to crawler coverage of “all URLs for all users” [3].

A proposition on the metric’s resolution Define $S_{\text{input}} = 1 - FP_{\text{inputs}}/N_{\text{inputs}}$ and $S_{\text{scenario}} = 1 - N_{\text{failed}}/N_{\text{scenarios}}$, where FP_{inputs} counts false alarms over all generated inputs and N_{failed} the scenarios in which the failure is uniform.

Proposition 1. Fix $f \geq 1$ failed scenarios, each producing a fixed number $a \geq 1$ of false-alarm inputs, and $s \geq 1$ successful scenarios each contributing m additional, alarm-free inputs. Then $S_{\text{input}} = 1 - (fa)/(fa + sm)$, which tends to 1 as $m \rightarrow \infty$, while $S_{\text{scenario}} = 1 - f/(f + s)$ is independent of m and strictly less than 1.

Proof. $N_{\text{inputs}} = fa + sm$ and $FP_{\text{inputs}} = fa$; substitution gives $S_{\text{input}} = 1 - (fa)/(fa + sm) \rightarrow 1$ as $m \rightarrow \infty$, whereas

the scenario-level figure depends only on the fixed counts f and s . $\square$

Corollary 1. No fixed input-level specificity threshold $t < 1$ can guarantee detection of a scenario-uniform failure class: choosing $m > (fa/s) \cdot t/(1 - t)$ drives S_{input} above t while the f failing scenarios remain failing.

The metric is therefore not merely imprecise: it is arbitrarily dilutable with respect to a scenario-uniform failure—adding unaffected inputs drives it toward 1 without touching the failure—so it lacks stable scenario-level resolution.

Second implementation An independently written implementation of the same predicates was compared with the main one on a subset fixed before the comparison, not sampled after it: the in-band scenarios on which a second reading is defined, holding the exception-policy variant at the representative level and the reachability reading at the one each relation uses (MR-002 blind, MR-004 aware). This yields 30 (configuration, scenario) points; MR-002 is defined on 30 of the resulting relation comparisons and MR-004 on the remaining 24, giving 54 in all. The two agreed on 54 of them (1.00; Wilson 95% CI $[0.934, 1.000]$). Both were written from the same reading of the semantics, so this bounds implementation accident only; it does not bound a shared blind spot, and we do not present it as corroboration.

5. Real systems: Gitea, Gogs, GitLab, and a seeded control

A synthetic system cannot tell us whether the crawl-time reading we identified is an artefact of our construction, so we repeated the essential comparison on real systems. We chose Gitea 1.22.6 for genuine object-level sharing with per-object visibility, a machine-readable sharing surface, and a documented administrative grant path. Three boundaries on the experiment’s conclusions, stated before the numbers. First, on the endpoints and authorization transitions exercised in this study, Gitea enforced the expected decisions: no instance of the studied vulnerability was found, so the experiment measures only the false-proof side. Second, the cell study uses our instantiation of the predicates, not the engine; the engine’s own relations are executed separately in Section 6.5. Third, the reachability model is reported under

Table 2

The grant surface of Gitea 1.22.6 (instance-independent).

quantity	count
paths in the system's API description	253
grant endpoints	15
revocation endpoints	2
of which usable without site-admin	8

Table 3

Oracle-readability sampling protocol on Gitea. Each row is a distinct population and denominator, so the rows are not comparable: both the vantage and the unit of the denominator differ.

vantage	attempts	readable
owner / administrator	56	56
arbitrary subject (black box)	40	0
subject also owner/admin (representative grid)	56	16

both readings—crawl-time snapshot (what the definitions presuppose) and live at execution time—so the responsible variable can be identified. The experiment produces 96 cells, 48 per reading.

Grant surface Table 2 counts an endpoint as grant-capable if, without modifying the system, a request can move a (subject, object) pair from not-entitled to entitled. The 8 endpoints drivable by a non-administrator are the out-of-band channel in its real form: a subject who is not an administrator can create an entitlement that a crawl of that subject's own interface need not reveal. The cross-vendor table below reports a normalised census (semantically equivalent routes merged, non-authorization grant surface excluded), which is why its Gitea figure (13) differs from the raw count (15) here; the two answer different questions.

Is the oracle readable, and from where? The oracle's readability depends entirely on the vantage, so the counts below use different populations and must not be read as one denominator. From the owner/administrator vantage the sharing surface is readable on 56 of 56 attempts: complete. From a black-box subject's own vantage—the vantage a black-box tool occupies—it is readable on 0 of 40: never. An intermediate count of 16/56 comes only from subjects who happen to be owners or administrators, and does not constrain the mechanism; it is therefore not synonymous with “an arbitrary black-box subject can read the oracle”. Table 3 collects these populations. On 48 of 48 applicable cells the credential-substitution check confirmed that our model agrees with the system's behaviour.

The mismatch is in time, not in visibility Table 4 is the load-bearing result: the adjudication departs from the real authorization state on 10 of 48 cells under the crawl-time

reading and 0 of the same cells under the live reading—same objects, same entitlements, same predicates. Under the crawl-time reading, a bookkeeping adjudication produces 10 false proofs on the 25 authorized cells (rate 0.4000), against 0 alarms under the live reading; v3 produces 0 false proofs, and of the controls `direct_only` again over-claims (0 false proofs from 23 alarms) while `direct_observe` emits 0. Channel provenance of the authorized access (24 of 25 not observable by the framework log, 0.9600) is a property of our topology—a design parameter, not a field prevalence.

An endogenous control The most serious alternative explanation is that public objects are simply harder to enumerate—visibility, not time. Two objects that end equally public but became public at different moments control for this (Table 5): the predicate reports unreachability on 5 of 6 cells only for the object made public after the crawl, and on 0 for the object public throughout, with the MR firing correspondingly (5 vs. 0). Final visibility is identical; the responsible variable is when the model was read.

5.1. Cross-vendor replication: Gogs 0.14.3 and GitLab 17.11.7

We repeated the identical protocol on two further systems (Table 6): **Gogs** 0.14.3, the project Gitea forked from in 2016 (related lineage, independently evolved code; census from its pinned route table, 80 paths), and **GitLab** 17.11.7 (no lineage; census from the pinned running instance's routes, 1357 paths, since its OpenAPI document omits core grant endpoints). Gogs is a replication across implementations, GitLab across vendors within one domain.

The replication changes one thing and confirms two. What it changes concerns the oracle: *readability does not imply soundness*. Gitea's owner-vantage surface is readable and sound (agrees with actual access on all 48 cells). Gogs's collaborator-membership endpoint is equally readable (48/48) but unsound—wrong on 17 cells (35.4%), exactly the authorizations arising outside direct membership: site administration (8), public visibility (5), out-of-band grants (4). GitLab's `members/all/{u}` resolves inherited membership but public visibility and site administration confer no membership: 18 false negatives (37.5%). What the oracle cannot see is predictable from what the surface measures—why clause 2 of the admissibility condition (Section 6.6) cannot be automated from readability alone. The consequence is measurable: on Gitea the oracle-gated adjudicators produced no false proofs (0 v3, 0 direct-observe), while on Gogs they manufacture them (8 snapshot, 5 live, 17 direct-observe) and on GitLab again (5, 18), every v3 false proof falling on a visibility-channel cell. A readable oracle with a blind spot converts its own blind spot into confirmed violations; no admissible oracle was available from the tested introspection surfaces on Gogs or GitLab, so the honest adjudicator abstains on the stratum where those surfaces would license a promotion.

Table 4

Departure from the real authorization state, per reading of the reachability model.

reading of the reachability model	mismatches	cells
crawl-time snapshot (as the definitions presuppose)	10	48
live at execution time	0	48

Table 5

Endogenous control: two objects that are both public at the end.

object	cells	cannotReach true	MR fires
public before the crawl	6	0	0
public only after the crawl	6	5	5

Two results replicate everywhere: bookkeeping alarms false-proof on legitimately granted access (10/11/10 cells) with the same out-of-band-dominant grant profile; and the reachability mismatch persists (10/19/10 snapshot). On Gogs the mismatch only partially resolves live (13 remain—team or public grants never enter a subject’s personal crawl); on GitLab the live reading recovers everything (0 remain)—a second, vendor-specific failure of the snapshot premise, in the crawl model rather than the oracle.

5.2. A positive control with a seeded vulnerability

On the tested endpoints and transitions the three real systems enforce the expected decisions, so the question remains: *when a real object-level flaw is present, does the same detector find it?* We answer with a positive control. A *controlled authorization target* faithfully emulates the Gitea API contract the engine consumes, with a correct internal

authorization model and 3 seeded vulnerable instances—all instantiating one fault pattern (a none-entitlement subject served the object at GET /api/v1/repos/{owner}/{repo}), differing in subject and object, not in fault location; Table 7 is therefore evidence about this fault pattern, not three independent defect discoveries. Two design points: the target deliberately exposes its authorization state to the subject-side harness (mirroring the admissibility condition), which production Gitea does not—on the seeded target the subject-side harness can read the authorization state of every cell by design, whereas on production Gitea the oracle is readable from the owner/admin vantage (56/56) but from an arbitrary subject’s vantage it is not (0/40); and the introspection surface reports the *correct* none permission, so the oracle stays sound and the defect is visible only as a discrepancy between oracle and request. We drove it with the same topology, mutation, crawl, introspection and adjudication code as above, differing only in the system under test; ground truth comes from the seed list, not from the detector.

Table 6

Cross-vendor replication under the identical three-phase protocol (crawl, out-of-band authorization change, adjudicate) on Gitea 1.22.6, Gogs 0.14.3, and GitLab 17.11.7. Counts are per reading of the same 48-cell grid; the live reading is given in parentheses where both exist. The snapshot reading is the operating point of the published relations. Gitea and Gogs share lineage (fork, 2016); GitLab shares none. The team-derived row is read out of the per-system oracle false negatives by channel (Section 5.1). The Gitea grant count here is the *normalised* M1 census (semantically equivalent routes merged, non-authorization grant surface excluded), and therefore differs from the raw route count in Table 2; the two tables use different normalisations and are not in conflict.

	Gitea 1.22.6	Gogs 0.14.3	GitLab 17.11.7
API entries in pinned version (M1 denominator)	253	80	1357
Grant endpoints (M1)	13	10	18
of which usable without site admin	10	4	17
Owner-vantage oracle readable	48	48	48
oracle agrees with actual access	48	31	30
oracle wrong (false negatives)	0	17	18
oracle wrong: team-derived access missed	0	4	0
Subject-vantage oracle readable	8	35	48
Reachability mismatch, snapshot (live)	10 (0)	19 (13)	10 (0)
False proofs: bookkeeping alarm, snapshot (live)	10 (0)	11 (5)	10 (0)
False proofs: v3 (oracle-gated), snapshot (live)	0 (0)	8 (5)	5 (0)
False proofs: direct-only (oracle only)	0 (0)	17 (17)	18 (18)
False proofs: direct-observe (oracle $\wedge$ request)	0 (0)	17 (17)	18 (18)

Table 7

Positive control: detection of 3 seeded vulnerable instances (one fault pattern). TP/FN/FP/TN are counted against the known seed, not against a model.

method	TP	FN	FP	TN
mr_bookkeeping	3	0	2	43
v3 (owner vantage)	3	0	0	45
v3 (subject vantage)	3	0	0	45
direct_only	3	0	30	15
direct_observe	3	0	0	45
direct_observe (subject)	3	0	0	45

The observation-gated adjudicator behaves exactly as the negative result predicts (Table 7): `direct_observe` and `v3` catch all 3 planted instances with zero false positives; `direct_only` again over-claims (30 false positives on cells where the oracle says none but no access succeeds). The control narrows the claim to this fault pattern: when a sound oracle is available, the metamorphic step is not necessary for detection here—both `direct_observe` and `v3` catch all 3 instances, while `direct_only` again over-claims. It does not generalise beyond one fault pattern, and the seeded BFLA defect (below) remains outside the read-oriented logic’s coverage. The control also delimits the detector: we seeded one BFLA defect (non-privileged bob succeeding at PUT . . . /collaborators/{u}, HTTP 204), which the read-oriented logic does not cover—a known boundary, reported as such rather than as a detection failure. This is a controlled research artefact, not production Gitea; it validates the detector’s logic, not Gitea’s CVE posture. Its source, seed list and run script are in the reproducibility package at <https://github.com/ubfbuz3/JISA> (tag artifact-v1).

6. Three boundaries, confirmed at source, and executed

6.1. Boundary 1 (RQ1): deterministic false proofs as designed

Taken as published—raw alarms, or alarms filtered by bookkeeping exceptions—the adjudication reports a legitimate access granted out of band after the crawl as an already-established violation, in one direction. On the constructed system, rate 1.00 (8 of 8 alarms) against 0 for the same share created in band; on Gitea, 10 false proofs on 25 authorized cells (0.4000) under the crawl-time reading, against 0 alarms live—the error is a property of the *reading*, not of the system. The direction is forced: the predicate has exactly two representations, and a grant created outside the crawled interface has no representation but “cannot reach”. The predecessor attributes its residual false alarms to crawler coverage [3]; we add that the failure is a function of *when* the model was read, which is what makes the Boundary-2 repair available—and empty.

6.2. Boundary 2 (RQ2): with a sound, readable oracle, the metamorphic step adds no adjudication information

If the false proofs of Boundary 1 are removed by consulting a readable introspection surface, it is the oracle that carries the work: for the authorization relations and scenarios evaluated here, replacing the entire metamorphic step with a single ordinary request—one that consumes the same readable oracle and access outcome—provides no additional measurable detection benefit.

We state the reason rather than present it as a discovered effect, because it is a necessary condition on the claim, not an empirical regularity.

Lemma 1 (oracle-sufficient adjudication). Let O be an authorization oracle that is *readable* (its answer can be obtained) and *sound* (its answer equals the true permission), and let R be the observed outcome of an ordinary request. Any adjudicator whose verdict is a function $V = f(O, R)$ of O and R alone is extensionally equal to the direct consumer $D := f(O, R)$: the two return the same verdict on every input. Consequently a metamorphic step can be *shown* to be necessary only by exhibiting an observable it consumes that the direct consumer does not.

Proof. An adjudicator of the stated form and D are the same function applied to the same pair of arguments, so they agree pointwise; the last sentence is the contrapositive. $\square$

The lemma is a requirement on *claims*, not a verdict on metamorphic testing: it says only that a verdict computable from (O, R) cannot be shown to beat the direct consumer of (O, R) . The evaluated relations do consult a third quantity—the crawl-time reachability model M —so the empirical question is whether M carries authorization information that a readable, sound O does not already supply. For such an O , M is either concordant with the true state, in which case it adds no information O lacks, or discordant, in which case it is false by construction (Boundary 1). Source inspection confirms the two relations read only M and the response (Section 6.4); the experiment confirms the consequence.

Evidence: Table 1—0 false proofs vs. 0 (a tie at zero, not a win), recall 0.50 vs. 0.49, difference 0.0139, 95% CI $[-0.0797, 0.1072]$ —descriptive, not a test; no margin was fixed in advance. Gitea independently reproduces the zero-false-proof result (0 vs. 0), but does not provide a recall comparison because the tested surface contains no seeded vulnerability. Two qualifications are part of the claim: the surviving difference is on recall alone, and there the MR-based method fails on all 3 disagreements; and the control is *constructed*, not discovered—`direct_observe` instantiates exactly the D of the lemma, so the tie is the empirical face of the necessary condition, a statement about the definitions, not a coincidental measurement.

6.3. Boundary 3 (RQ3): the reported specificity is not conditioned on a reachability-authorization check, and uses a low-resolution denominator

The reported 99.81% is a statement about a corpus of generated inputs; it is not conditioned on an explicit validation that the reachability model coincides with the true authorization state. The omission: adjudication is a function of the reachability model, so where model and true state diverge—the out-of-band case of Boundary 1—the figure measures agreement with the tool’s own crawler; the authors concede the mapping is “not always available” [2], but the number is reported without the condition. The denominator: a failure uniform across a scenario class contributes only as many false positives as that class emits inputs, and is diluted by the rest of the run (Proposition below). We are not saying the figure is false or the false positives misdiagnosed; the claim is that such a metric cannot discharge its own premise nor see a class-uniform failure.

6.4. Implementation-level confirmation

Definitions can be read generously, so we checked the predicates at source in the public library [2]. It ships 23 relations, 11 authorization relations, 8 gating on the two predicates analysed. Four implementation-level properties not made explicit in the main presentation: (i) *the freeze is hard-coded*—cannotReachThroughGUI is set-membership over the crawl inputs, issues no request, and is cached in a field written once and never invalidated (userCanRetrieveContent likewise reads a crawl-produced store loaded once); (ii) *the clause that could have caught it is inert*—the “absent across all users” set is built by stopping each recorded sequence at the first login action, so it is always empty; (iii) *missing data defaults to an alarm*—with an empty or null crawl argument the predicate returns “cannot reach”, so an empty crawl satisfies the precondition: the mechanism behind Boundary 1’s deterministic direction; (iv) *the role relation is configuration, not state*—the supervisor guard reads a static configuration file and reports true for identical accounts, so it cannot reflect a later grant.

6.5. Executing the published relations

The library ships relations but no execution environment; we supplied the system configuration and crawl snapshot and ran the relations—source unmodified, build configuration (pom.xml) and an external harness added (1 changed upstream file, 0 changed Java sources, two added classes totalling 612 lines)—against Gitea 1.22.6 (Table 8). 10 of 11 authorization relations executed; 2 returned failures, 6 alarms in all—the engine’s own getFailures() value (Table 9). What the run establishes is scoped to the *reachability-gated subset that fires here*, not a full replay of the catalogue: one relation aborts on an engine-side defect, most issue no request because their own guards are unsatisfied, and the content predicate is never invoked. Almost none of the 1483.7 seconds is analysis: each leaf evaluation invokes a

Table 8

Execution facts for the native run of the published relations (Section 6.5). All values are emitted by ingest_native_engine.py from the run record; upstream Java sources are unchanged apart from the build configuration.

item	value
authorization relations: catalogue / executed / fired	11 / 10 / 2
engine alarms (the engine’s own return value)	6
HTTP requests attributed to observing relations	270
changed upstream Java sources	0
build/config changes (one pom.xml line)	1
added harness classes / added lines	two / 612

browser-cleanup hook costing minutes, a cost of the missing environment that bounds any reproduction.

The load-bearing cell: the subject bob holds a legitimate entitlement to alice/s1, granted out of band after the crawl. There the published cannotReachThroughGUI answers “cannot reach”—the answer it is *defined* to give, since the subject’s crawl record has no entry for the repository; across the run it answered so 1149 of 7321 invocations (15.7%), 5 on this cell, and 2 relations (OTG_AUTHZ_002, OTG_AUTHZ_002c) report a failure there. The access is authorized: all 3 granted cells were re-probed, and the grantee’s response hashes identically to the owner’s while a third account is refused. The 6 alarms fall on 3 cells, all post-crawl grants, none on a non-entitled object. Limits: the harness’s HTTP observation channel is exact for the reachability predicate (set-membership, no request) but unexercised for the content predicate (0 invocations; supervisor guard 8642, 1321 true); not all gated relations fire, for reasons inside their own guards, and 1 abort on an engine-side defect—so the reproduced false proof is that of the reachability-gated subset that fires, not a full-replay count of the published relations. And Gitea, on the tested surface, enforces the expected decisions, so the run can only exhibit false proofs.

6.6. Admissibility, and falsification

Three properties must be kept apart. A surface is *readable* when its answer can be obtained from the vantage in question; *sound* when its answer equals the true authorization state; and *admissible*, for promotion to violation-proven, when it is readable *and* one can give a reason that it is not of the same kind as the defect under test—not itself a reachability artefact whose blind spot correlates with the blindness tested. Readability is what makes soundness usable, but it neither implies soundness nor confers admissibility: the owner-vantage surface is readable on every cell in each system (48 Gitea, 48 Gogs, 48 GitLab), yet sound on Gitea alone—it agrees with actual access on 48, 31 and 30 of those cells respectively. The cross-vendor systems are therefore not cases of “no oracle”: a readable membership surface exists on both, but it is

Table 9

The published authorization relations executed against Gitea 1.22.6. Each row is the applicability chain from the catalogue to the outcome: *status* is the engine's per-relation result (10 of 11 evaluated, 1 aborted on an engine-side defect), *alarms* its own `getFailures()` return (2 relations, 6 in all), and *requests* (270 total) attributed to the relation observing them first.

MR	status	alarms	requests
OTG_AUTHZ_001	executed	0	0
OTG_AUTHZ_001b	executed	0	0
OTG_AUTHZ_001b2	executed	0	0
OTG_AUTHZ_002	executed	3	270
OTG_AUTHZ_002a	executed	0	0
OTG_AUTHZ_002b	executed	0	0
OTG_AUTHZ_002c	executed	3	0
OTG_AUTHZ_002d	executed	0	0
OTG_AUTHZ_002e	error	–	–
OTG_AUTHZ_003	executed	0	0
OTG_AUTHZ_004	executed	0	0

unsound for the authorization channels exercised here, so it fails clause (2). Clause 2 is a per-system judgement and not automatable from readability alone; the cross-vendor results make it concrete (Gogs and GitLab fail it; their adjudicators convert the blind spots into 8 and 5 false proofs). Where the condition holds, the honest method consumes the oracle directly and drops the relation; where it does not, the honest method abstains—as v3 does on the unreadable stratum, where the aggregate table would have ranked it best, a ranking that reflects abstention rather than improved detection. Each boundary would fall as follows: Boundary 1, if the predicates were defined over live rather than recorded reachability; Boundary 2, by an authorization MR needing an observable that a succeeding request plus a readable oracle cannot produce; Boundary 3, by a specificity in scenario units, or with the condition explicit, that still fails to separate the configurations studied here.

7. Threats to validity

Source versus execution We read the predicates at source (built under JDK 8 and Maven; Section 6.4) and then executed the relations, unmodified apart from the build configuration and our harness (Section 6.5). The harness substitutes an HTTP observation channel—exact for the reachability predicate, not exercised for the content predicate, whose invocation count we report rather than claim cleanliness we did not demonstrate. The run converts one class of claim (the false proof is produced by the *published* relations on a real system); it does not convert the rest: the scenario/configuration and cell studies remain statements about our instantiation, our exception vocabulary belongs to the same predicate family, and we report no sensitivity or specificity attributable to MST-wi.

Metric and statistics The specificity approximation uses two follow-up inputs per case—not the engine's count; the conclusion about the denominator's resolution does not depend on the exact value. Records within a design cell are not

independent (Section 3), so all intervals are descriptive and no result rests on statistical significance.

Two MRs, one family We instantiated 2 MRs of one family—relations gated on the absence of evidence. The native run lifts the count to 10 of 11 catalog authorization relations but not the family: only the reachability-gated ones alarm in our run. This is the paper's single most important limitation; raising it requires relations gated otherwise, a different experiment.

Real systems Three systems (one lineage pair, one independent), chosen for accessibility, none vulnerable on the tested surface—so only the false-proof side is measured, within one domain, the git forge. Channel proportions are topology design parameters, not field prevalence. Constructed scenarios with construction-defined ground truth are why the real-system section exists; three systems tested once each are still not a population.

Our own earlier claims An earlier iteration proposed v3 as a mechanism on the strength of its precision (1.00), which was bought with 111 false negatives, and a control (`direct_only`, precision 0.50) that discarded the access result. Replacing it with a fair control produced the negative result. We record this as the condition any similar claim must meet: a control consuming the same observable quantity as the ground truth.

8. Related work

Metamorphic security testing Metamorphic testing was brought to web security by Mai et al. [3], who cast security test patterns—authentication, authorization, session—as metamorphic relations so that faults can be exposed without an oracle. The line matured into MST-wi [2], whose 76 system-agnostic relations over a Web-specific domain language are the object of our study. We propose no new relation and no better detector; we take the technique's own admission that the role-to-resource mapping is “not always

available” at face value and follow it to its consequence for what the adjudication can decide. A relation whose guard reads a crawl-time model is sound only while the model equals the authorization state, and that equality is exactly what the technique cannot check for itself.

Detecting broken access control A large and active literature detects access-control flaws directly, and it is worth separating the oracles it relies on. Black-box probing drives requests with and without the rights of another subject and looks for a missing refusal [4, 5], a line that runs back to AUTHSCOPE [6] and that has been extended to deployed systems under ethical constraints [7]. A complementary, white-box line recovers the *enforced* policy from the implementation—by reverse-engineering RBAC policies [8], by statically extracting authorization policies [9] or database access-control rules [10], or by locating the permission checks themselves [11]—and then flags violations against that recovered model: BOLA in database-backed applications [12], missing-owner-check and function-level flaws in Java web applications [13, 14], and privilege escalation in polyglot microservices [15]. Specification-driven API fuzzers generate the requests well but inherit the same oracle difficulty [16–18]; adding explicit access-policy violation checks [19] moves them toward the detection problem but does not remove the need for a ground truth, and a recent turn delegates generation or probing to language models [20–22]. Our question is orthogonal to all of these. Each reports detections against an oracle it holds privately—a policy, a seeded defect, a differential response; we ask what an existing oracle-substituting technique *decides* when the entitlement information it needs is not among its observables, and we make that a measurable property of the technique rather than of a system.

LLMs and the oracle problem LLMs are used for vulnerability detection and specification inference [23–26]; we use none in the detection loop. The transferable condition this paper exposes is that any claim of necessity must be tested against a fair substitute that consumes the same observable quantity as the ground truth.

Measurement and evaluation Our contribution is finally a measurement, and it connects to work on the quality of the artefacts security studies are built on—for instance the observation that vulnerability datasets are frequently mislabelled, duplicated or otherwise unfit for the claims drawn from them [27]. The artefact we report is of the same family but inside a study rather than a dataset: a specificity figure whose *denominator* is structurally unable to register a whole class of failure, and whose operating point silently assumes a reachability model equals the authorization state. We therefore read a testing technique’s own reported metric as a measurement instrument, stating explicitly what its denominator can and cannot resolve.

9. Conclusion

For object-level authorization, MST-wi’s predicates do not know whether the acting subject is entitled to the object; they know what the interface, as crawled, permitted. Instantiating them as defined produced three boundaries. As designed, the adjudication false-proofs on legitimately granted access, deterministically (1.00 constructed; 0.4000 Gitea), because an out-of-band grant fits only the wrong one of the predicate’s two representations. The natural repair—a readable, sound authorization oracle—works, and in that regime the evaluated metamorphic step adds no adjudication information the oracle and the request do not already carry: this is a necessary condition (Lemma 1), whose empirical face is that a single ordinary request yields the same false-proof count (false proofs 0 vs. 0) and the same recall (difference 0.0139, CI [−0.0797, 0.1072]; descriptive, no margin pre-specified); the one surviving difference is on recall, where on all 3 disagreeing records the MR-based method fails. On Gitea the comparison is limited to false-proof behaviour, because no vulnerability was present on the tested surface. On the real system the mismatch appears only under the crawl-time reading (10/48, vs. 0 live), with an endogenous control holding final visibility fixed: the responsible variable is time, not visibility. And the reported specificity of 99.81% is not conditioned on an explicit check that the reachability model equals the authorization state, and is computed in units that cannot resolve a scenario-uniform failure. Reading the implementation confirmed the semantics and added the mechanism: the crawl model is frozen by a cache never invalidated, and a missing crawl defaults to “unreachable”. The positive control closes the loop: on a seeded vulnerable target the oracle-gated adjudicator detects all planted instances with zero false positives.

What we do not claim We do not claim that MST-wi is wrong or say anything about its accuracy as a detector; the native run establishes a qualitative outcome, for one gate family, on one real system that on the tested endpoints and transitions enforces the expected decisions. Channel proportions are design parameters; our two MRs are two, of one family, and that is the ceiling on everything we say; and we have not found a better detector.

What remains expensive Moving from characterisation to something stronger needs, first, a real system that *contains* the flaw, so detection can be measured; and second, relations whose gate is not the absence of evidence—checkable against a subject’s entitlements without pretending that entitlements are reachable facts. The transferable condition—we call it the *same-observable necessity criterion*—is the one that killed our own mechanism: a component’s necessity must be tested against a substitute that consumes the *same* observable quantity as the ground truth; a substitute that consumes less does not establish incremental value.

Declarations

Data availability

The raw records of the studies reported here—the scenario/configuration study, the real-system cell study, the execution of the published relations, and the seeded positive control—together with the scripts that compute every number in this paper from those records, are publicly available at <https://github.com/ubfbuz3/JISA> (tag artifact-v1, which pins the artifact and is re-anchored to the submission commit). The manuscript source is in the same repository. Running `bash reproduce.sh` regenerates all figures, tables and the paper PDF from the raw records.

Competing interests

The authors declare no competing interests, financial or otherwise.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Authors' contributions

Both authors contributed to the work. Qiulong Lin and Nanlei Chen jointly developed the conceptualisation and methodology, designed and implemented the software and analysis pipeline, carried out the investigation and formal analysis, and wrote the manuscript.

Ethics

Not applicable. This study involved no human participants, no personal data, and no animals. All experiments were performed against software systems that we deployed and controlled locally.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used Claude (Anthropic) to assist with literature verification, the development of the analysis scripts, and language editing. Where it contributed to analysis code, every generated or modified script was reviewed, executed, and its output checked by the authors against the raw records. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article. All experimental computations and final reported values were independently checked by the authors against the released scripts and raw records.

References

- [1] OWASP Foundation, API1:2023 Broken Object Level Authorization, OWASP API Security Top 10, accessed 18 September 2026 (2023). URL <https://api-security.owasp.org/editions/2023/en/0xa1-broken-object-level-authorization/>
- [2] N. B. Chaleshtari, F. Pastore, A. Goknil, L. Briand, Metamorphic testing for web system security, *IEEE Transactions on Software Engineering* (2023) 3430–3471 doi:10.1109/tse.2023.3256322. URL <https://doi.org/10.1109/icst46399.2020.00028>
- [3] P. X. Mai, F. Pastore, A. Goknil, L. Briand, Metamorphic security testing for web systems, in: 2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST), 2020, pp. 186–197. doi:10.1109/icst46399.2020.00028. URL <https://doi.org/10.1109/icst46399.2020.00028>
- [4] F. Liu, Y. Zhang, E. Li, W. Meng, Y.-Q. Shi, Q. Wang, M. Yang, Bacscan: Automatic black-box detection of broken-access-control vulnerabilities in web applications, in: *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, 2025. doi:10.1145/3719027.3744825. URL <https://doi.org/10.1145/3719027.3744825>
- [5] I. P. A. Dharmaadi, M. Alhanahnah, V.-T. Pham, F. Mohsen, F. Turkmen, Bacfuzz: Exposing the silence on broken access control vulnerabilities in web applications, *ArXiv* (2025). doi:10.48550/arxiv.2507.15984. URL <https://doi.org/10.48550/arxiv.2507.15984>
- [6] C. Zuo, Q. Zhao, Z. Lin, Authscope: Towards automatic discovery of vulnerable authorizations in online services, in: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017. doi:10.1145/3133956.3134089. URL <https://doi.org/10.1145/3133956.3134089>
- [7] S. E. H. Chehade, F. Hantke, B. Stock, 403 forbidden? ethically evaluating broken access control in the wild, in: 2025 IEEE Symposium on Security and Privacy (SP), 2025, pp. 3218–3235. doi:10.1109/sp61157.2025.00252. URL <https://doi.org/10.1109/SP61157.2025.00252>
- [8] H. Le, L. K. Shar, D. Bianculli, L. Briand, D. C. Nguyen, Automated reverse engineering of role-based access control policies of web applications, *Journal of Systems and Software* 184 (2022) 111109. doi:10.1016/j.jss.2021.111109. URL <https://doi.org/10.1016/j.jss.2021.111109>
- [9] B. Berger, R. W. Nguempnang, K. Sohr, R. Koschke, Static extraction of enforced authorization policies seeauthz, in: 2020 IEEE 20th International Working Conference on Source Code Analysis and Manipulation (SCAM), 2020, pp. 187–197. doi:10.1109/scam51674.2020.00026. URL <https://doi.org/10.1109/SCAM51674.2020.00026>
- [10] W. Zhang, D. Bali, J. Kerney, A. Panda, S. Shenker, Extracting database access-control policies from web applications, in: 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI '26), 2026, pp. 249–270. URL <https://www.usenix.org/conference/osdi26/presentation/zhang-wen>
- [11] Y.-Q. Shi, F. Liu, G. Yang, Y. Zhang, Y. Cao, E. Li, S. Chen, Facilitating access control vulnerability detection in modern java web applications with accurate permission check identification, *IEEE Transactions on Information Forensics and Security* (2025) 10683–10698 doi:10.1109/tifs.2025.3614424. URL <https://doi.org/10.1109/TIFS.2025.3614424>
- [12] Y. Huang, C. Shi, J. Lu, H. Li, H. Meng, L. Li, Detecting broken object-level authorization vulnerabilities in database-backed applications, in: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024. doi:10.1145/3658644.3690227. URL <https://doi.org/10.1145/3658644.3690227>
- [13] F. Liu, Y.-Q. Shi, Y. Zhang, G. Yang, E. Li, M. Yang, Mocguard: Automatically detecting missing-owner-check vulnerabilities in java web applications, in: 2025 IEEE Symposium on Security and Privacy (SP), 2025, pp. 903–919. doi:10.1109/sp61157.2025.00010. URL <https://doi.org/10.1109/SP61157.2025.00010>
- [14] R. Feng, M. Zhang, A. Xia, J. Cheng, Y. Li, Y. Zhang, Y. Liu, Jauthguard: Automatic detection for broken access control in java web apis, in: 2025 IEEE International Conference on Systems, Man, and Cybernetics (SMC), 2025, pp. 3102–3107. doi:10.1109/smc58881.

2025.11343625.

URL <https://doi.org/10.1109/SMC58881.2025.11343625>

- [15] P. Li, H. Chong, Y. Cao, J. Yang, Detecting privilege escalation in polyglot microservices via agentic program analysis, in: 2026 IEEE Symposium on Security and Privacy (SP), 2026, pp. 1747–1765. doi:10.1109/sp63933.2026.00121.
URL <https://doi.org/10.1109/SP63933.2026.00121>
- [16] V. Atlidakis, P. Godefroid, M. Polishchuk, Restler: Stateful rest api fuzzing, in: 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), 2019, pp. 748–758. doi:10.1109/icse.2019.00083.
URL <https://doi.org/10.1109/ICSE.2019.00083>
- [17] A. Arcuri, M. Zhang, J. P. Galeotti, Advanced white-box heuristics for search-based fuzzing of rest apis, ACM Transactions on Software Engineering and Methodology (2024). doi:10.1145/3652157.
URL <https://doi.org/10.1145/3652157>
- [18] S. Seran, M. Zhang, O. Duman, A. Arcuri, Handling web service interactions in fuzzing with search-based mock-generation, ACM Transactions on Software Engineering and Methodology (2026). doi:10.1145/3731558.
URL <https://doi.org/10.1145/3731558>
- [19] Ömür Şahin, M. Zhang, A. Arcuri, Enhancing rest api fuzzing with access policy violation checks and injection attacks, Journal of Systems and Software 243 (2027). doi:10.1016/j.jss.2026.113060.
URL <https://doi.org/10.1016/j.jss.2026.113060>
- [20] M. A. Uddin, S. Weerasinghe, C. Wojtak, T. Cerný, D. Silva-Junior, M. E. S. Ribeiro, A. S. Abdelfattah, Automated generation of microservice authorization tests using large language models, in: 2026 International Conference on Service-Oriented System Engineering (SOSE), 2026, pp. 41–50. doi:10.1109/sose71128.2026.00014.
URL <https://doi.org/10.1109/SOSE71128.2026.00014>
- [21] J. Sun, R. Tian, S. Zhao, X. Zhou, H. Wang, Y. Zhang, Llm-assisted idor detection in hospital mini-programs: Risks to pii and phi, in: 2025 IEEE 24th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), 2025, pp. 2078–2086. doi:10.1109/trustcom66490.2025.00242.
URL <https://doi.org/10.1109/Trustcom66490.2025.00242>
- [22] E. Wang, W. Xie, S. Li, R. Liu, Y. Zhou, Z. Wang, B. Wang, Large language model-powered protected interface evasion: Automated discovery of broken access control vulnerabilities in internet of things devices, Sensors (Basel, Switzerland) (2025). doi:10.3390/s25092913.
URL <https://doi.org/10.3390/s25092913>
- [23] H. Wang, Z. Wang, P. Liu, A hybrid llm workflow can help identify user privilege related variables in programs of any size, ArXiv (2024). doi:10.48550/arxiv.2403.15723.
URL <https://doi.org/10.48550/arxiv.2403.15723>
- [24] C. Wen, J. Cao, J. Su, Z. Xu, S. Qin, M. He, C. Tian, Enchanting program specification synthesis by large language models using static analysis and program verification, in: International Conference on Computer Aided Verification, 2024, pp. 302–328. doi:10.1007/978-3-031-65630-9_16.
URL https://doi.org/10.1007/978-3-031-65630-9_16
- [25] R. Wang, H. Zhuang, H. Cao, S. Chen, Measuring the reasoning boundaries of large language models for implicit security invariants in code: A controlled empirical study, in: 2026 IEEE 50th Annual Computers, Software, and Applications Conference (COMPSAC), 2026, pp. 3146–3157. doi:10.1109/compsac69091.2026.00467.
URL <https://doi.org/10.1109/COMPSAC69091.2026.00467>
- [26] T. Le-Cong, B. Le, T. Murray, Can llms reason about program semantics? a comprehensive evaluation of llms on formal specification inference, in: Annual Meeting of the Association for Computational Linguistics, 2025, pp. 21991–22014. doi:10.18653/v1/2025.acl-long.1068.
URL <https://aclanthology.org/2025.acl-long.1068/>
- [27] R. Croft, M. A. Babar, M. M. Kholoosi, Data quality for software vulnerability datasets, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), 2023, pp. 121–133. doi:10.1109/icse48619.2023.00022.